

Generative AI: Transforming Code Generation

Generative AI (GenAI) is revolutionizing software development by acting as an **intelligent coding partner** that can write, suggest, optimize, document, and test code. Instead of manually writing everything, developers now **collaborate with AI** to accelerate delivery and improve quality.

1. How Generative AI Transforms Code Generation

Generative AI models (like GPT-based systems and GitHub Copilot) understand:

- Programming languages
- Coding patterns
- Software architecture
- Developer intent (from comments & prompts)

Key Transformations

Area	Traditional Way	With Generative AI
Writing Code	Manual typing	AI suggests code
Debugging	Search errors	AI explains errors
Documentation	Written later	Auto-generated
Testing	Manual creation	AI-generated
Refactoring	Time-consuming	One-command rewrite

2. Code Completion and Suggestions

GenAI predicts the next lines of code based on:

- File context
- Comments
- Function name
- Project structure

Example

Developer types:

```
def calculate_discount(price, percentage):
```

AI Suggests:

```
discount = price * (percentage / 100)
return price - discount
```

Benefit

- Reduces typing
 - Prevents syntax errors
 - Improves consistency
-

3. Code Completion and Suggestions: Example

Prompt (Comment)

```
// create a function to validate email
```

AI Output

```
function validateEmail(email) {
  const pattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return pattern.test(email);
}
```

4. Demo:

Building a To-Do List Application Using GitHub Copilot

Tech Stack

- HTML
 - CSS
 - JavaScript
-

Step 1: Create HTML

```
<input id="taskInput" placeholder="Enter task">
<button onclick="addTask()">Add</button>
<ul id="taskList"></ul>
```

Copilot auto-suggests structure.

Step 2: JavaScript Logic

```
function addTask() {  
  const task = document.getElementById("taskInput").value;  
  const li = document.createElement("li");  
  li.textContent = task;  
  document.getElementById("taskList").appendChild(li);  
}
```

Outcome

Fully working To-Do app created with minimal manual typing.

5. Custom Script Generation and Example

AI can generate scripts for automation.

Prompt

Create a Python script to rename all files in a folder with numbers.

Generated Script

```
import os  
  
files = os.listdir("files")  
  
for index, file in enumerate(files):  
    os.rename(f"files/{file}", f"files/{index}.txt")
```

6. Demo: Automating Boilerplate Code Generation with GitHub Copilot

Example – Flask App

Type Comment

```
# create flask app with home route
```

Copilot Suggests

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Welcome"

if __name__ == "__main__":
    app.run(debug=True)
```

Result

Instant backend skeleton.

7. Automated Documentation

GenAI generates:

- Function docstrings
 - API documentation
 - README files
-

8. Automated Documentation and Example

Function

```
def add(a, b):
    return a + b
```

AI Generated Docstring

```
def add(a, b):
    """
    Adds two numbers.

    Args:
        a (int): First number
        b (int): Second number

    Returns:
        int: Sum of a and b
    """
```

9. Demo: Building a Grocery App with GitHub Copilot and Flask

Features

- Add grocery item
 - View items
-

Flask App

```
from flask import Flask, request, jsonify

app = Flask(__name__)
items = []

@app.route("/add", methods=["POST"])
def add_item():
    items.append(request.json)
    return "Item Added"

@app.route("/items")
def get_items():
    return jsonify(items)

app.run(debug=True)
```

Copilot helps generate routes.

Outdated Code Refactoring

GenAI modernizes legacy code.

1. Outdated Code Refactoring and Example

Old Python

```
f = open("data.txt")
data = f.read()
f.close()
```

AI Refactored

```
with open("data.txt") as f:  
    data = f.read()
```

Test Case Generation

GenAI generates unit & integration tests.

Test Case Generation and Example

Function

```
def multiply(a, b):  
    return a * b
```

AI Test

```
def test_multiply():  
    assert multiply(2, 3) == 6
```

Demo: Generating Component Architecture and Building a Structured Flask Blog Using GitHub Copilot

Folder Structure

```
blog_app/  
├── app.py  
├── models.py  
├── routes.py  
├── templates/  
└── static/
```

Copilot generates base files and routes.

Demo: Creating a Pretrained Model to Generate an Application Structure

Conceptual Steps:

1. Collect code samples
2. Train model on project templates
3. Provide prompt:

Generate Django ecommerce app structure

4. Model outputs folders & files.
-

Quick Check

Q1. What is the main benefit of GenAI in coding?

- ✓ Faster development
-

Q2. Which tool works inside IDE?

- ✓ GitHub Copilot
-

Q3. What can GenAI generate?

- A) Code
- B) Tests
- C) Docs
- D) All

- ✓ D
-

Q4. Fix Code

```
def div(a,b):  
    return a/b
```

Improved

```
def div(a,b):  
    if b == 0:  
        return None  
    return a/b
```

Q5. Short Answer

Why still review AI code?

✓ Security & correctness.

☐ Summary

Generative AI transforms coding by providing:

- Smart suggestions
- Automation
- Faster prototyping
- Better quality

It acts as a **co-developer**, not a replacement.